



Student Name: Bhavya
Branch: CSE(AI & ML)
Semester:4
Subject Name: Database Management System

UID: 24BAI70791
Section/Group: 24AIT_KRG-1/G2
Subject Code: 24CSH-298

Experiment – 9

1. Aim of the Session

To create and implement PL/SQL packages by developing a package specification and package body containing procedures and shared cursors, in order to achieve modular, reusable, and efficient database programming.

2. Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

3. Objectives

To design and implement a PL/SQL package that includes procedures and shared cursors for structured and modular program development.

4. Procedure of the Experiment

1. Create an Employee table to store employee details like ID, name, salary, and department.
2. Insert sample records into the table to test the package functionality.
3. Create the package specification to declare procedures that will be used externally.
4. Create the package body to define the actual implementation of the declared procedures.
5. Declare a shared cursor inside the package to retrieve employee records efficiently.
6. Implement the procedure to open the cursor, fetch records, and display them.
7. Compile the package to check and remove any syntax errors.
8. Execute the package procedure using a PL/SQL block.
9. Verify the output by displaying employee details using DBMS_OUTPUT.

5. Practical / Experiment Steps

PROGRAM CODE:

-- Create Employee Table

```
CREATE TABLE Employee (
```

```
    emp_id NUMBER,
```

```
    emp_name VARCHAR2(50),
```

```
    salary NUMBER,
```

```
    dept VARCHAR2(50)
```

```
);
```

-- Insert Sample Data

```
INSERT INTO Employee VALUES (101, 'Rahul', 50000, 'IT');
```

```
INSERT INTO Employee VALUES (102, 'Sneha', 60000, 'HR');
```

```
INSERT INTO Employee VALUES (103, 'Amit', 55000, 'Finance');
```

-- Package Specification



CREATE OR REPLACE PACKAGE emp_pkg AS

 PROCEDURE display_employees;

END emp_pkg;

/

-- Package Body

CREATE OR REPLACE PACKAGE BODY emp_pkg AS

 CURSOR emp_cursor IS

 SELECT * FROM Employee;

 PROCEDURE display_employees IS

 emp_record emp_cursor%ROWTYPE;

 BEGIN

 OPEN emp_cursor;

 LOOP

 FETCH emp_cursor INTO emp_record;

 EXIT WHEN emp_cursor%NOTFOUND;

 DBMS_OUTPUT.PUT_LINE(

 'ID: ' || emp_record.emp_id ||

 ' Name: ' || emp_record.emp_name ||

 ' Salary: ' || emp_record.salary ||

 ' Dept: ' || emp_record.dept

);

 END LOOP;



```
CLOSE emp_cursor;
```

```
END;
```

```
END emp_pkg;
```

```
/
```

```
-- Execute Package
```

```
BEGIN
```

```
emp_pkg.display_employees;
```

```
END;
```

```
/
```

6. Input / Output Details and Screenshot

The screenshot shows a SQL IDE interface with a dark theme. The top bar has tabs for 'Query' and 'Query History'. The 'Query' tab is active, showing a SQL script with line numbers 1 through 8. The script is as follows:

```
1 DROP TABLE IF EXISTS employee;  
2  
3 CREATE TABLE Employee (  
4     emp_id INT,  
5     emp_name VARCHAR(50),  
6     salary INT,  
7     dept VARCHAR(50)  
8 );
```

Below the query editor, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying the following message:

```
CREATE TABLE  
  
Query returned successfully in 147 msec.
```

```

9
10 INSERT INTO Employee VALUES (101, 'Rahul', 50000, 'IT');
11 INSERT INTO Employee VALUES (102, 'Sneha', 60000, 'HR');
12 INSERT INTO Employee VALUES (103, 'Amit', 55000, 'Finance');
13

```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 619 msec.

```

13
14 SELECT * FROM employee;

```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1

	emp_id integer	emp_name character varying (50)	salary integer	dept character varying (50)
1	101	Rahul	50000	IT
2	102	Sneha	60000	HR
3	103	Amit	55000	Finance

```

15
16 CREATE OR REPLACE FUNCTION display_employees()
17 RETURNS VOID AS $$
18 DECLARE
19     emp_record RECORD;
20 BEGIN
21     FOR emp_record IN SELECT * FROM employee LOOP
22         RAISE NOTICE 'ID: %, Name: %, Salary: %, Dept: %',
23             emp_record.emp_id,
24             emp_record.emp_name,
25             emp_record.salary,
26             emp_record.dept;
27     END LOOP;
28 END;
29 $$ LANGUAGE plpgsql;

```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 398 msec.

```
30
31 SELECT display_employees();
```

Data Output	Messages	Notifications
NOTICE: ID: 101, Name: Rahul, Salary: 50000, Dept: IT		
NOTICE: ID: 102, Name: Sneha, Salary: 60000, Dept: HR		
NOTICE: ID: 103, Name: Amit, Salary: 55000, Dept: Finance		
Successfully run. Total query runtime: 169 msec.		
1 rows affected.		

7. Learning Outcome

- Understand the concept and structure of PL/SQL packages.
- Differentiate between package specification and package body.
- Develop procedures to perform database operations.
- Use cursors to retrieve and process records efficiently.
- Write modular and reusable PL/SQL code.